

Потоки

Общая информация

Общая информация

Пото́к выполне́ния (англ. *thread* — нить) — наименьшая единица обработки, исполнение которой может быть назначено ядром операционной системы.

Общая информация

Поток выполнения находится внутри процесса.

Несколько потоков выполнения могут существовать в рамках одного и того же процесса и **совместно использовать ресурсы**, такие как память, тогда как процессы не разделяют этих ресурсов.

Общая информация

Процесс содержит как минимум один (первичный) поток.

Общая информация

- ❑ Создание потоков требует от ОС меньших накладных расходов, чем процессов.
- ❑ В отличие от процессов, все потоки одного процесса всегда принадлежат одному приложению, поэтому ОС изолирует потоки в гораздо меньшей степени, нежели процессы в традиционной мультипрограммной системе.
- ❑ Все потоки одного процесса используют общие файлы, таймеры, устройства, одно и то же адресное пространство

Общая информация

Диспетчеризация – это переключение процессора с одного потока на другой.

Процесс диспетчеризации:

- ☐ Сохранение контекста потока, который требуется сменить.
- ☐ Загрузка контекста нового потока.
- ☐ Запуск нового потока на выполнение.

Общая информация

В большинстве OS используется вытесняющий алгоритм диспетчеризации.

Общая информация

- В основе многих вытесняющих алгоритмов планирования лежит концепция квантования. В соответствии с этой концепцией каждому потоку поочередно для выполнения предоставляется ограниченный непрерывный период процессорного времени — квант.

Общая информация

- Смена активного потока происходит, если:
 - ☐ - поток завершился;
 - ☐ - произошла ошибка;
 - ☐ - поток перешел в состояние ожидания;
 - ☐ - исчерпан квант процессорного времени, отведенный данному потоку.

Общая информация



Общая информация

- У каждого потока есть свой приоритет

Операционная система планирует к исполнению более приоритетные потоки или выделяет приоритетным потокам больший квант времени.

Общая информация

- Потоки получают приоритеты на базе классов приоритета своих процессов.

Общая информация

При создании потока его приоритет по умолчанию устанавливается равным приоритету процесса.

Приоритеты потоков могут принимать значения в интервале ± 2 относительно базового приоритета процесса.

Общая информация

При создании потока его приоритет по умолчанию устанавливается равным приоритету процесса.

Приоритеты потоков могут принимать значения в интервале ± 2 относительно базового приоритета процесса.

Общая информация

Уровни приоритета потоков (в порядке убывания):

- Highest
- Above normal
- Normal
- Below normal
- Lowest

ПОТОКИ

Пространство имен `System.Threading`

Пространство имен System.Threading

- Классы, поддерживающие многопоточное программирование, определены в пространстве имен System.Threading.

Пространство имен System.Threading

- ❑ **Interlocked** Этот тип обеспечивает элементарные операции для переменных, которые совместно используются несколькими потоками.
- ❑ **Monitor** Этот тип обеспечивает синхронизацию потоковых объектов, используя блокировки и ожидания / сигналы. Ключевое слово C# **lock** использует скрытый объект Monitor.
- ❑ **Mutex** Этот примитив синхронизации можно использовать для синхронизации между границами домена приложения.
- ❑ **ParameterizedThreadStart** Этот делегат позволяет потоку вызывать методы, которые принимают любое количество аргументов.

Пространство имен System.Threading

- ❑ **Semaphore** Этот тип позволяет вам ограничить число потоков, которые могут одновременно обращаться к ресурсу или определенному типу ресурса.
- ❑ **Thread** Этот тип представляет поток, который выполняется в CLR. Используя этот тип, вы можете создавать дополнительные потоки в исходном домене приложения.
- ❑ **ThreadPool** Этот тип позволяет вам взаимодействовать с поддерживаемым CLR пулом потоков в данном процессе.
- ❑ **ThreadPriority** Это перечисление представляет уровень приоритета потока (Самый высокий, Нормальный и т.д.).

Пространство имен System.Threading

- ❑ **ThreadStart** Этот делегат используется для указания метода, вызываемого для данного потока. В отличие от делегата `ParameterizedThreadStart`, цели `ThreadStart` всегда должны иметь один и тот же прототип.
- ❑ **ThreadState** Это перечисление определяет допустимые состояния, которые может принимать поток (работает, прерван и т.д.).
- ❑ **Timer** Этот тип предоставляет механизм для выполнения метода через определенные промежутки времени.
- ❑ **TimerCallback** Этот тип делегата используется вместе с типами `Timer`.

ПОТОКИ

Класс Thread

Класс Thread

- Класс **Thread** представляет объектно-ориентированную оболочку для заданного потока выполнения.

Класс Thread

- Класс **Thread** также определяет ряд методов (как статических, так и на уровне экземпляров класса), которые позволяют создавать новые потоки в приложении, а также приостанавливать, возобновлять, останавливать и уничтожать определенный поток.

Методы и свойства класса Thread

- ❑ **Sleep()** - Этот метод (статический) приостанавливает текущий поток на указанное время. Когда поток приостановлен, он не использует процессорное время.
- ❑ **IsAlive** Возвращает логическое значение, которое указывает, был ли этот поток запущен (и еще не завершен или не прерван).
- ❑ **Name** Позволяет установить понятное текстовое имя темы.

Методы и свойства класса Thread

- ❑ **Priority** Получает или задает приоритет потока, которому может быть присвоено значение из перечисления ThreadPriority.
- ❑ **ThreadState** Получает состояние этого потока, которому может быть присвоено значение из перечисления ThreadState.
- ❑ **Abort()** Указывает CLR прекратить поток.
- ❑ **Join()** Блокирует вызывающий поток до тех пор, пока не закончится указанный поток (тот, в котором вызывается Join ()).

Методы и свойства класса Thread

- ❑ **Resume()** Возобновляет поток, который был ранее приостановлен.
- ❑ **Start()** Указывает CLR выполнить поток.
- ❑ **Suspend()** Приостанавливает поток. Если поток уже приостановлен, вызов Suspend() не имеет никакого эффекта.

Класс Thread. Создание и запуск потока

- Конструктор потока:
- ***public Thread(ThreadStart точка_входа)***
-
- *точка_входа* — это имя метода, вызываемого с целью начать выполнение потока
- *ThreadStart* — делегат, определенный в среде .NET Framework:
-
- ***public delegate void ThreadStart ()***

Класс Thread. Создание и запуск потока

- Созданный поток ***не начнет выполняться*** до тех пор, пока не будет вызван его метод **Start()**.

Класс Thread. Создание и запуск потока

```
• class MyThread
• {
•     public int Count;
•     string thrdName;
•     public MyThread(string name)
•     {
•         Count = 0;
•         thrdName = name;
•     }
• }
```

Класс Thread. Создание и запуск потока

```
•      /// <summary>
•      /// Функция потока
•      /// </summary>
•      public void Run()
•      {
•          Console.WriteLine(thrdName + " начал.");
•          do
•          {
•              // Длительное вычисление
•              Thread.Sleep(500); // ждать 500 мс
•              Console.WriteLine($"В потоке {thrdName}, Count = {Count}");
•              Count++;
•          }
•          while (Count < 10);
•          Console.WriteLine(thrdName + " завершен.");
•      }
•  }
• }
```

Класс Thread. Создание и запуск потока

- `Console.WriteLine("Основной поток начат.");`
- `// Создать объект типа MyThread.`
- `MyThread mt = new MyThread("Потомок #1");`
- `// Создать поток из метода Run объекта MyThread.`
- `Thread newThrd = new Thread(mt.Run);`
- `// Начать выполнение потока.`
- `newThrd.Start();`

- `// параллельно выводить индикацию о выполнении`
- `do`
- `{`
- `Console.Write(".");`
- `Thread.Sleep(100);`
- `}`
- `while (mt.Count != 10);`

- `Console.WriteLine("Основной поток завершен");`

Класс Thread. Создание и запуск потока

- `Console.WriteLine("Основной поток начат.");`
- `// Создать объект типа MyThread.`
- `MyThread mt = new MyThread("Потомок #1");`
- `// Создать поток из метода Run объекта MyThread.`
- `Thread newThrd = new Thread(mt.Run);`
- `// Начать выполнение потока.`
- `newThrd.Start();`
- `// параллельно выводить индикацию о выполнении`
- `for (int i = 0; i < 20; i++)`
- `{`
- `Console.Write(".");`
- `Thread.Sleep(100);`
- `};`
- `newThrd.Join();`
- `Console.WriteLine("Основной поток завершен");`

Класс Thread. Создание и запуск потока

- Запуск потока при создании класса MyThread:

- `public MyThread(string name)`
- `{`
- `Count = 0;`
- `Thrd = new Thread(this.Run);`
- `Thrd.Name = name; // задать имя потока`
- `Thrd.Start(); // начать поток`
- `}`

Класс Thread. Создание и запуск потока

- Если потоку нужно передать параметры, то они передается потоку в следующей форме метода Start ():
- **public void Start(object arg)**

Класс Thread. Пул потоков

- Создание потоков требует времени. Если есть различные короткие задачи, подлежащие выполнению, можно создать набор потоков заранее и затем просто отправлять соответствующие запросы

Класс Thread. Пул потоков

- Многие приложения создают потоки, которые проводят много времени в спящем состоянии, ожидая возникновения события. Другие потоки могут переходить в спящее состояние только для того, чтобы периодически их пробуждать для опроса об изменении или обновлении информации о статусе. Пул потоков позволяет более эффективно использовать потоки, предоставляя вашему приложению пул рабочих потоков, которыми управляет система.

Класс Thread. Пул потоков

- Для управления пулом потоков используется класс **ThreadPool**

Класс Thread. Пул потоков

- Чтобы запросить обработку задачи потоком в пуле потоков, вызовите метод **QueueUserWorkItem**.
- Этот метод принимает в качестве параметра ссылку на **метод или делегат**, который будет вызываться потоком, выбранным из пула потоков.

Класс Thread. Пул потоков

- `// Постановка задачи в очередь`
- - `ThreadPool.QueueUserWorkItem(ThreadProc);`
- `// Процедура потока`
- `static void ThreadProc(Object stateInfo)`
- `{`
- - `// код задачи`
- `}`

Класс Thread. Пул потоков

- Пул потоков управляет потоками эффективно, уменьшая количество создаваемых, запускаемых и останавливаемых потоков.

Класс Thread. Пул потоков (ограничения)

- ❑ Все потоки в пуле потоков являются фоновыми. Сделать поток из пула приоритетным не удастся.
- ❑ Нельзя изменять приоритет или имя находящего в пуле потока.
- ❑ Потоки в пуле подходят для выполнения только коротких задач. Если необходимо, чтобы поток функционировал все время (как, например, поток средства проверки орфографии в Word), его следует создавать с помощью класса Thread.
- ❑ Созданный поток невозможно прерывать или находить по имени.

Потоки

Синхронизация потоков

Синхронизация потоков

- Во многопоточных приложениях потоки могут использовать общие разделяемые ресурсы: процессорное время, память, файлы, переменные.

Синхронизация потоков

- Например: все потоки в домене приложений имеют одновременный доступ к общим данным приложения.
- Может возникнуть состояние гонок, когда поток А будет считывать данные, в то время как поток В будет изменять эти данные

Синхронизация потоков

- Какие данные прочитает поток A – до изменения или после? И какие из этих значений считать правильными? Дело в том, что мы точно не знаем, в какой последовательности планировщик потоков будет выделять кванты времени для каждого потока.

Синхронизация потоков

```
• public class ConcurrentThread
• {
•     Random r = new();
•     public void PrintNumbers()
•     {
•         for (int i = 0; i < 10; i++)
•         {
•             // Приостановить поток на случайное время
•             Thread.Sleep(100 * r.Next(5));
•             Console.Write($"{i}, ");
•         }
•         Console.WriteLine();
•     }
• }
```

Синхронизация потоков

- `var p = new ConcurrentThread();`
- `// Создать 10 потоков`
- `Thread[] threads = new Thread[10];`
- `for (int i = 0; i < 10; i++)`
- `{`
- `threads[i] =`
- `new Thread(p.PrintNumbers);`
- `}`
- `// Запустить все потоки`
- `foreach (Thread t in threads)`
- `t.Start();`
- `Console.ReadLine();`

Синхронизация потоков

- Порядок выполнения созданных потоков абсолютно непредсказуем

Синхронизация потоков

- Для синхронизации работы потоков используются разные объекты – Monitor, Mutex, Semaphore и др.

Синхронизация потоков (критическая секция)

- Ключевое слово ***lock*** позволяет определить область операторов (критическую секцию), которые должны быть синхронизированы между потоками.
- lock(lockObj)
- {синхронизируемые операторы }
-

Синхронизация потоков (Monitor)

- Конструкция оператора ***lock*** инкапсулирует в себе синтаксис использования мониторов.

Синхронизация потоков (Mutex)

- Класс **Mutex** является классом-оберткой над соответствующим объектом ОС Windows «Mutex»
- Особенностью мьютекса является возможность синхронизировать *процессы*. Для этого при создании объекта нужно задать мьютексу имя.

Синхронизация потоков (Semaphore)

- Класс **Semaphore** является классом-оберткой над соответствующим объектом ОС Windows «Semaphore»
- Semaphore работает аналогично классу Mutex. Отличие заключается в том, что Mutex разрешает доступ к ресурсу только одному потоку (процессу), в то время как Semaphore разрешает доступ к ресурсам сразу нескольким потокам (процессам). Количество указывается при создании объекта Semaphore.

Синхронизация потоков (Event)

- Класс **AutoResetEvent** является оберткой над объектом ОС Windows «Event» и позволяет переключить данный объект-событие из сигнального в несигнальное состояние.

Неблокирующая синхронизация потоков

- Класс **Interlocked** позволяет вызывать потокобезопасные простые атомарные операции с переменными без использования объектов синхронизации (Monitor, Mutex и т.д.)
- Применение класса **Interlocked** является гораздо более быстрым подходом по сравнению с остальными приемами по обеспечению синхронизации.

Неблокирующая синхронизация потоков

- Некоторые методы класса Interlocked:

Decrement(ref x)	Безопасно уменьшает значение x на 1. Эквивалентно <code>x--</code>
Exchange(ref x, value)	Безопасно присваивает x значение value. Эквивалентно <code>x=value</code>
Increment(ref x)	Безопасно увеличивает значение x на 1 Эквивалентно <code>x++</code>
Add(ref x, value)	Безопасно складывает значение x и значение value. Эквивалентно <code>x+=value</code>

Синхронизация потоков (System.Threading.Timer)

- Класс **Timer** из пространства имен **System.Threading** позволяет запускать метод по истечению заданного периода времени.

Синхронизация потоков (System.Timers.Timer)

- Класс **Timer** из пространства имен **System.Timers** позволяет генерировать события по истечению заданного периода времени.